



操作系统课程实验

——从0到1的小型内核实现

2025年 秋季学期

lab-2 内存管理初步

1. 物理内存：内存区域 + 内存分页
2. 物理内存：链式组织 + 分配回收
3. 内核态虚拟内存：页表 + 页表项
4. 内核态虚拟内存：内核页表
5. 实验小结

1. 物理内存：内存区域 + 内存分页

```
SECTIONS
{
    . = 0x80000000;

    .text : {
        *(.text .text.*)
        . = ALIGN(0x1000);
        /*
         * _trampoline = .;
         */
        *(trampsec)
        . = ALIGN(0x1000);
        ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
        /*
         * PROVIDE(KERNEL_DATA = .);
         */
    }

    .rodata : {
        . = ALIGN(16);
        *(.srodata .srodata.*) /* do not need to distinguish this from .rodata */
        . = ALIGN(16);
        *(.rodata .rodata.*)
    }

    .data : {
        . = ALIGN(16);
        *(.sdata .sdata.*) /* do not need to distinguish this from .data */
        . = ALIGN(16);
        *(.data .data.*)
    }

    .bss : {
        . = ALIGN(16);
        *(.sbss .sbss.*) /* do not need to distinguish this from .bss */
        . = ALIGN(16);
        *(.bss .bss.*)
    }

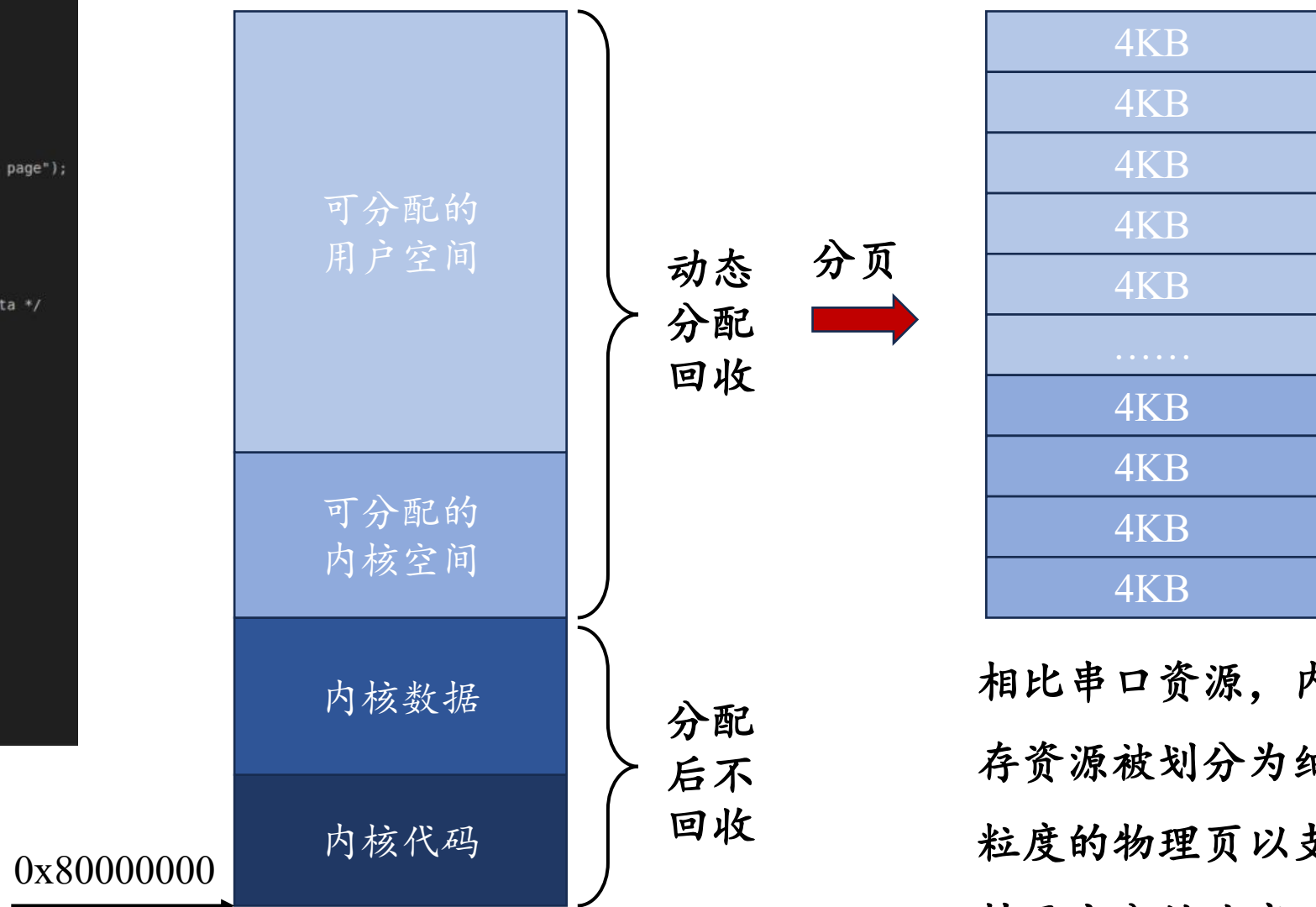
    . = ALIGN(4096);
    PROVIDE(ALLOC_BEGIN = .);
    PROVIDE(ALLOC_END = 0x80000000 + 128M);
}
```

内核程序：kernel-qemu.elf

➤ 代码区域：.text

➤ 数据区域：.rodata .data .bss

机器启动时的内存区域分布



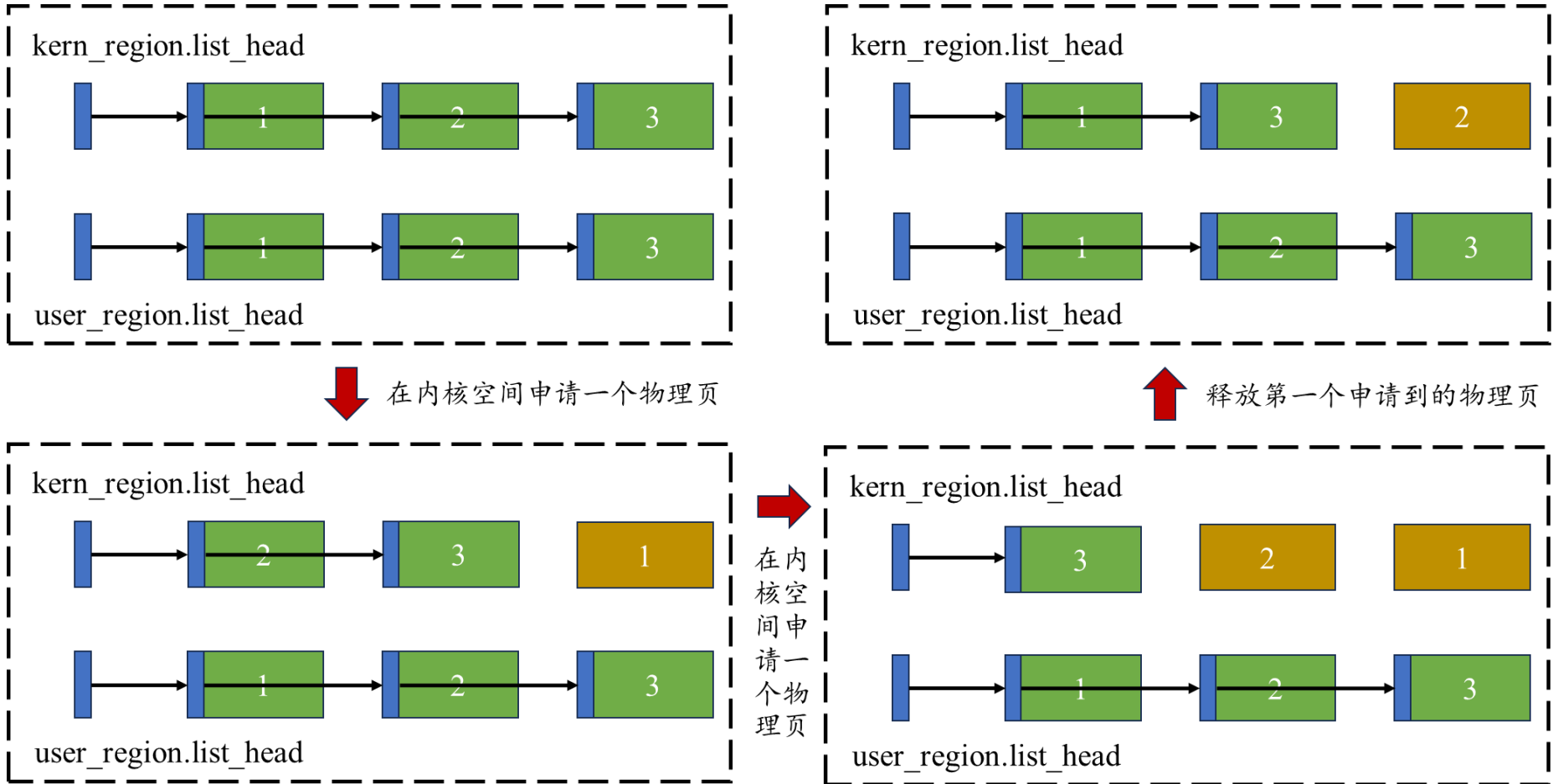
相比串口资源，内存资源被划分为细粒度的物理页以支持更充分的共享。

2. 物理内存：链式组织 + 分配回收

经过上一页的讨论，物理内存已经被抽象为两个4KB物理页数组（内核/用户）

然而，数组并不适合做快速的内存分配（需要遍历数组来寻找空闲页面）

因此，我们选择使用“带头节点的单链表”来描述和管理空闲物理页面



物理页申请

被翻译为
移除链表节点

物理页释放

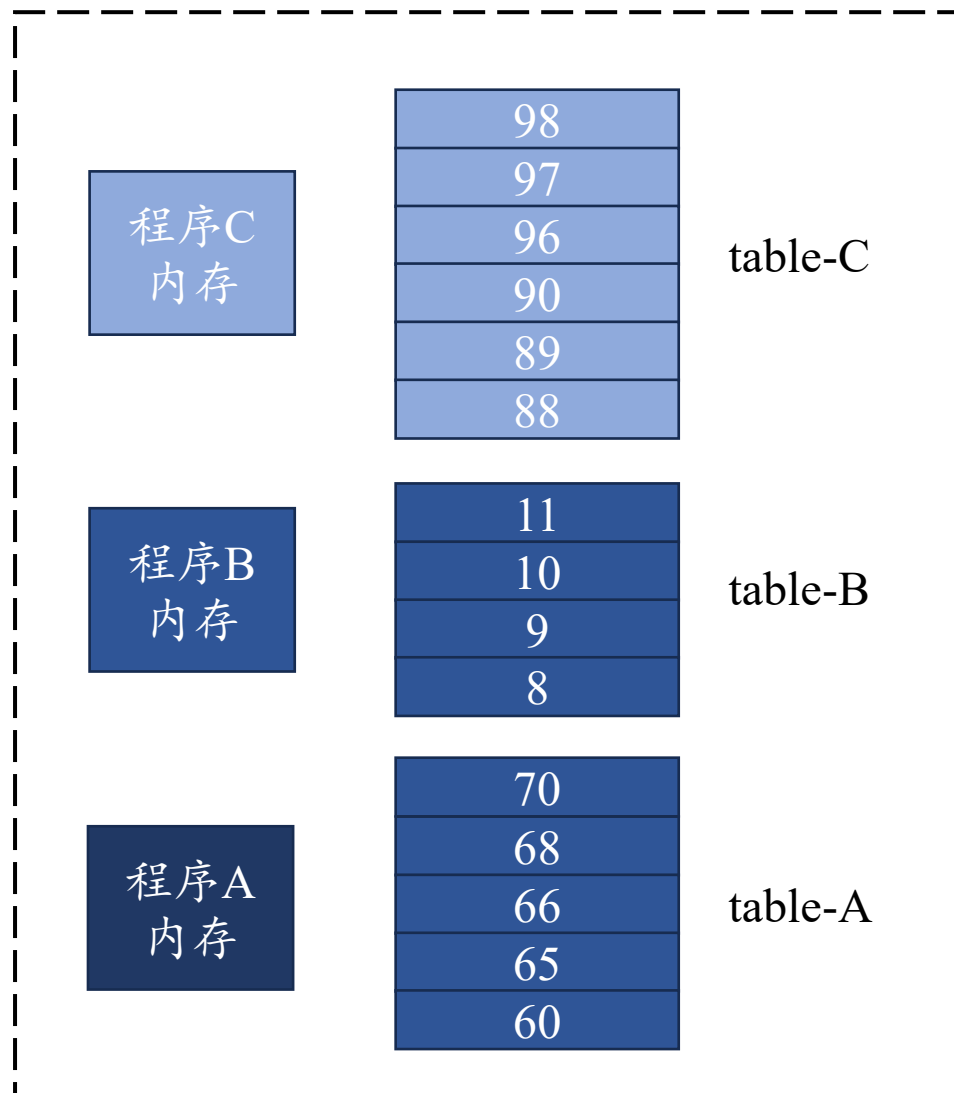
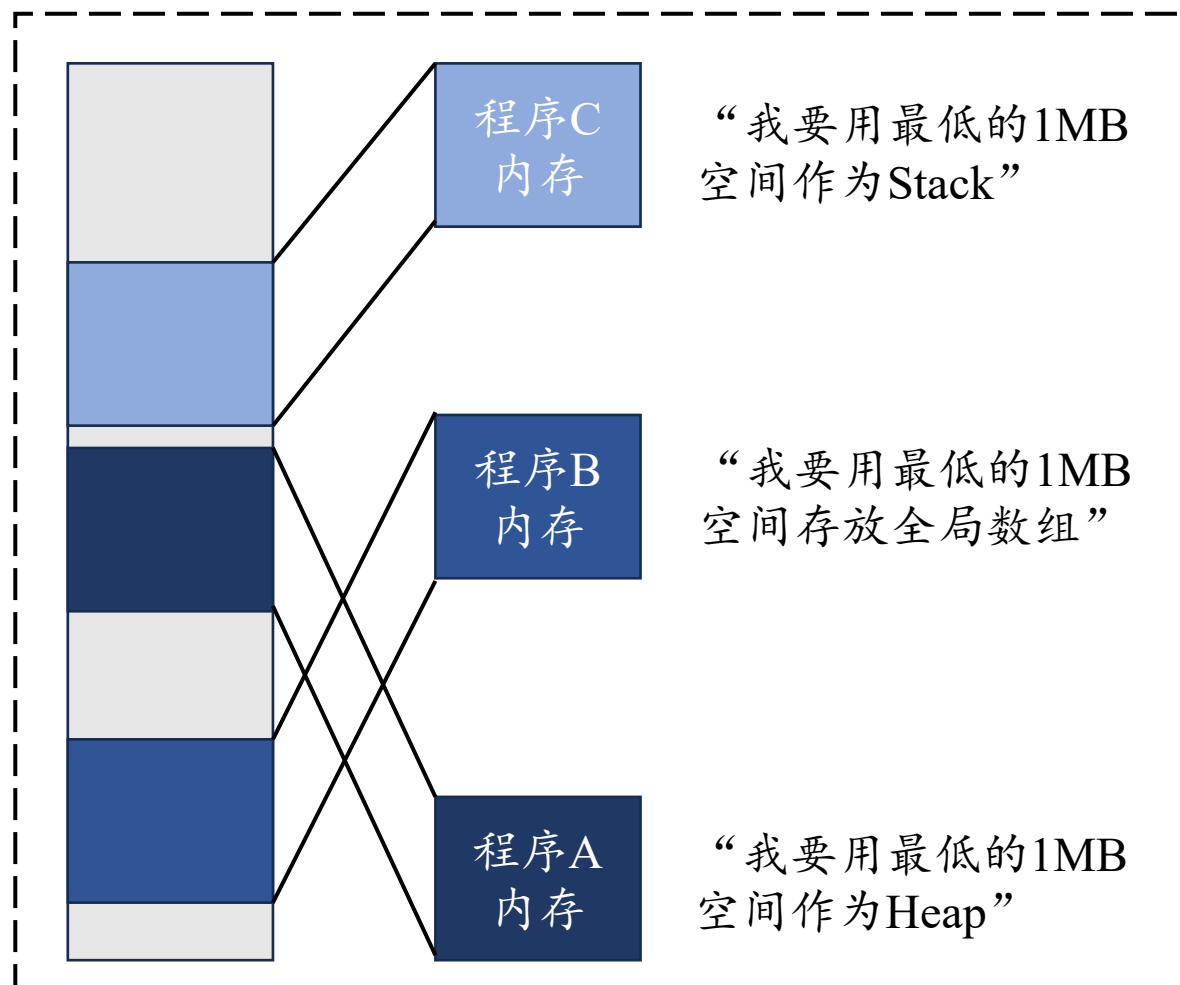
被翻译为
插入链表节点

注意：通过锁来保证上述操作原子性

3. 内核态虚拟内存：页表 + 页表项

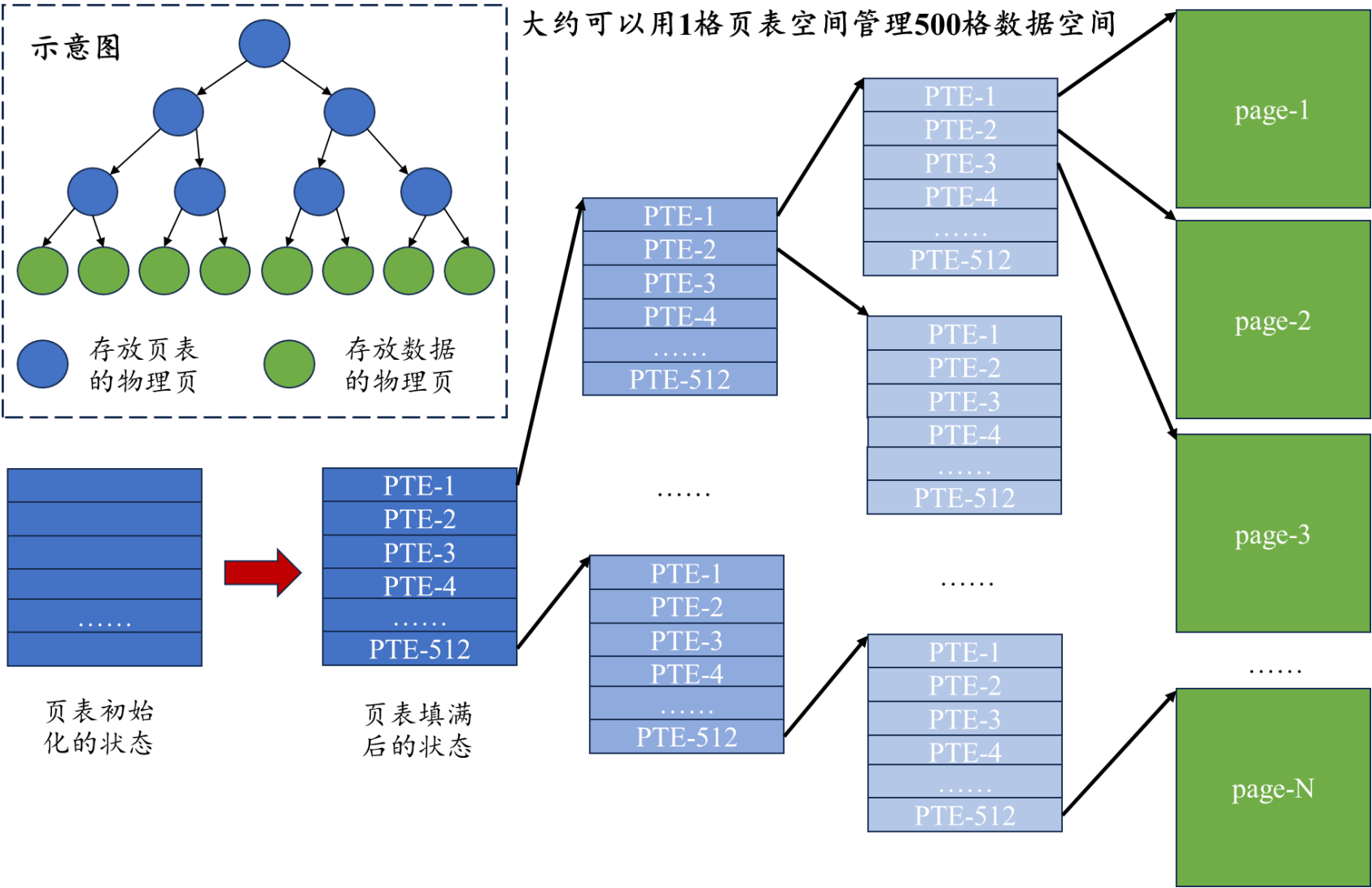
实现物理内存管理就够了吗？OR 为什么需要引入虚拟内存？

- 编程模型需要“独占内存”的假设
- 各个程序持有的内存资源需要一个清单做记录



3. 内核态虚拟内存：页表 + 页表项

综合两种需求，OS内核需要实现基于页表的虚拟内存管理！



- 绝大多数物理页用于存放数据，少数物理页用于存放页表。
- 页表负责记录虚拟地址对应的物理地址，而MMU负责自动地执行这个翻译过程。
- 将顶级页表的地址填入satp寄存器即可启动基于此页表的地址翻译。
- 段错误背后的原因：MMU查页表时遇到了空白或非法PTE。

4. 内核态虚拟内存：内核页表

作为一个程序，OS内核需要一张自己的内核页表

引入用户进程后，每个进程都有一张自己的用户页表

可以看得出来，内核页表的地位是与众不同的，体现在：

- 内核页表采用对等映射（虚拟地址 == 物理地址）
- 内核页表映射了所有通用内存空间（能访问任何一块内存）
- 内核页表映射了所有设备内存空间（能访问任何一个外设）

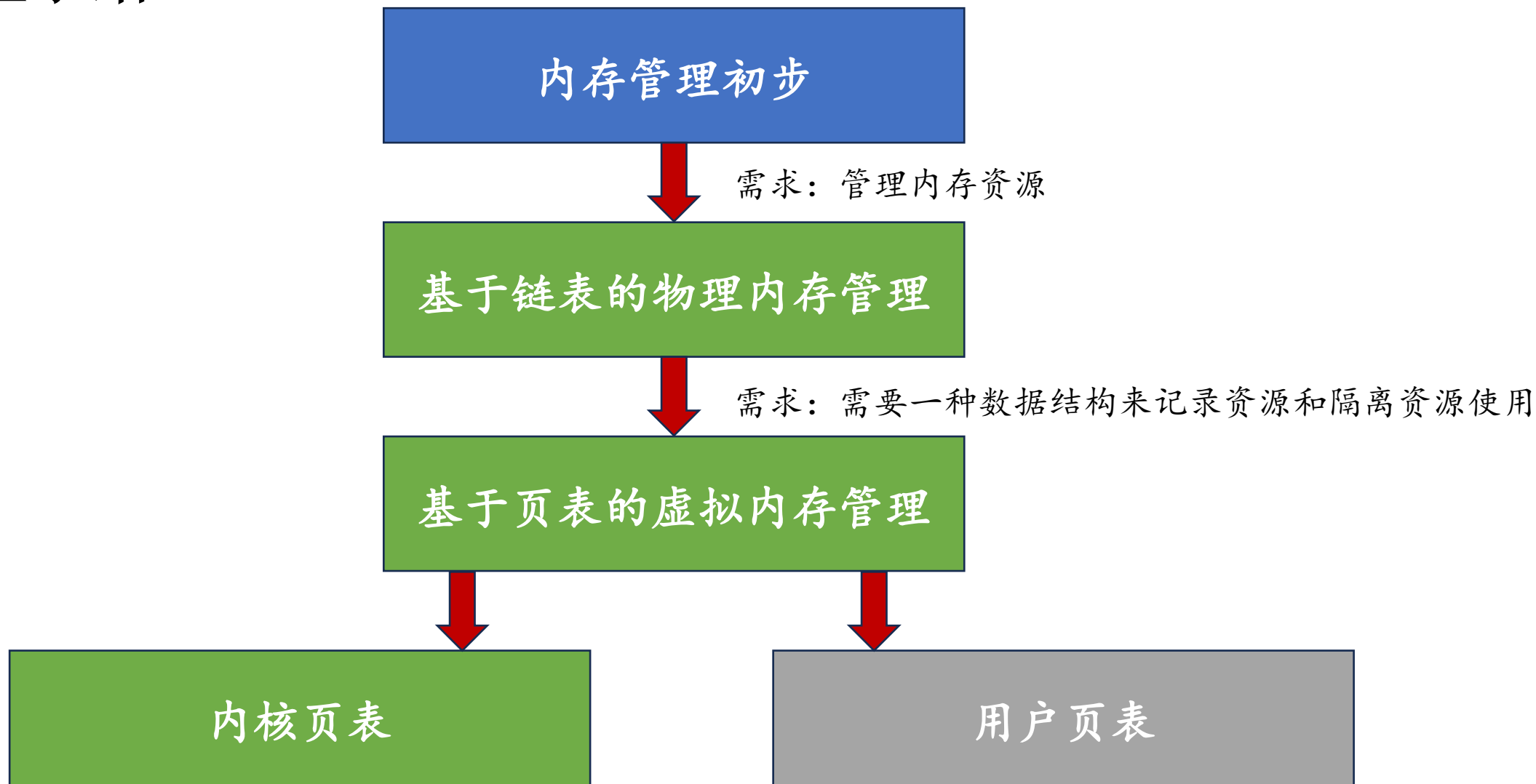
注意：每个CPU都有一套自己的寄存器

因此，填写satp寄存器的操作是每个CPU都需要做的事情

但是，各个CPU的执行流其实是共享一个内核地址空间（用同一张页表）

所以，内核页表的初始化只需要一个CPU来做就好

5. 实验小结



用户程序的需求驱动内核程序的发展，开发者要理解各个模块的逻辑关系和发展脉络